

Reviewer Pack — Gowers Uniformity Norm Gap in Read-Twice vs Read-Once BPs

Entry cd21f13f8cbc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 18:13:38 UTC

Gowers Uniformity Norm Gap in Read-Twice vs Read-Once BPs

Entry ID: cd21f13f8cbc **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 18:13:38 UTC

1. Conjecture

Field A (mathematical branch): Additive Combinatorics (Gowers Uniformity Norms) **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For any read-twice BP P with size S , the Gowers uniformity norm of order 3 of its characteristic function satisfies $\|f_P\|_{U^3} = O(\log S)$. For the IP_2 function, $\|f_{IP_2}\|_{U^3} = \Omega(n)$.

Rationale (proposer's reasoning):

Gowers norms capture higher-order correlations critical for functions like IP_2 , which require exponential size for read-once BPs. Read-twice BPs, with their limited memory, exhibit pseudorandomness reflected in bounded Gowers norms.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e9a9349e3b3dcad1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gowers_u3_norm(f, n):
        # Compute Gowers U^3 norm using discrete Fourier transform
        F = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(n + 1):
            for j in range(n + 1):
                for k in range(n + 1):
                    F[i][j] += f[(i, j, k)] * math.exp(-2j * math.pi * (i * k + j * k) / n)
        norm = sum(abs(F[i][j]) ** 4 for i in range(n + 1) for j in range(n + 1)) ** 0.25
        return norm

    def ip2_function(x):
        # IP_2 function: f(x, y) = x * y (mod n)
        n = len(x)
        f = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                f[i][j] = (i * j) % n
        return f

    def generate_read_twice_bp(n, S):
        # Generate a random read-twice BP with size S
        bp = {}
        for _ in range(S):
            x1 = random.randint(0, n - 1)
            x2 = random.randint(0, n - 1)
            y = random.randint(0, n - 1)
            if (x1, x2) not in bp:
                bp[(x1, x2)] = []
            bp[(x1, x2)].append(y)
        return bp

    n = 40
    S = 100

    # Generate read-twice BP and compute U^3 norm
    bp = generate_read_twice_bp(n, S)
    f_bp = {(i, j, k): random.randint(0, 1) for i in range(n) for j in range(n) for k in range(n)}
    u3_norm_bp = gowers_u3_norm(f_bp, n)
```

```

# Compute IP_2 function and its U^3 norm
f_ip2 = ip2_function([i for i in range(n)])
u3_norm_ip2 = gowers_u3_norm(f_ip2, n)

return {
    "metric_name": "Gowers U^3 norm",
    "metric_value": max(u3_norm_bp, u3_norm_ip2),
    "instances_tested": 1,
    "conjecture_holds": u3_norm_bp <= math.log(S) and u3_norm_ip2 >= n / 2,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_83aae375.py", line 78, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_83aae375.py", line 58, in run_trial
    u3_norm_bp = gowers_u3_norm(f_bp, n)
                  ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_83aae375.py", line 27, in gowers_u3_norm
    F[i][j] += f[(i, j, k)] * math.exp(-2j * math.pi * (i * k + j * k) / n)
                  ~~~~~
TypeError: must be real number, not complex

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to type error; no data produced to evaluate conjecture | next: Fix complex number handling in gowers_u3_norm implementation

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111695
2	propose	ollama_remote	qwen3:8b	0	0	64479
3	preregistration	ollama_remote	qwen3:8b	0	0	28094
4	novelty	ollama_remote	qwen3:8b	0	0	22901
5	novelty	ollama_remote	qwen3:8b	0	0	17275
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13994
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10122
8	judge	ollama_remote	qwen3:8b	0	0	17636

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 286195 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/cd21f13f8cbc.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/cd21f13f8cbc.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/cd21f13f8cbc.tar.gz (if generated)